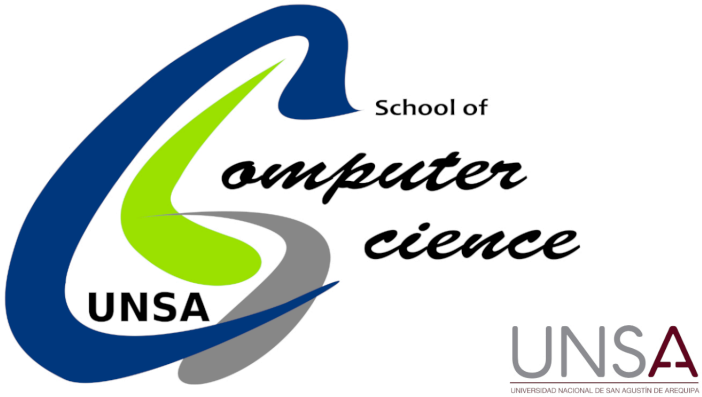
UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS



ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN
COMPUTACION PARALELA Y DISTRIBUIDA

LABORATORIO: OPENMP — LISTA ENLAZADA MULTITHREADING Y ODD-EVEN TRANSPOSITION SORT

INTEGRANTES:

- Lopez Zegarra, Ivan Alexander

DOCENTE:

ALVARO HENRY MAMANI ALIAGA

AREQUIPA – PERÚ
2026

Índice

1. Introducción y Entorno Experimental 1
    1.1. Propósito y Objetivos 1
    1.2. Entorno de Ejecución y Hardware 1
    1.3. Metodología de Medición 2
2. Parte I: Lista Enlazada Multithreading 2
    2.1. Diseño de la Estructura y Secciones Críticas 2
        2.1.1. Delimitación de Secciones Críticas 2
    2.2. Estrategias de Sincronización Evaluadas 2
    2.3. Pruebas de Correctitud y Verificación 3
    2.4. Evaluación de Rendimiento 4
        2.4.1. Discusión: El Caso del 99% de Lectura vs. 99% de Escritura 5
    2.5. Respuestas a las Preguntas de Análisis — Lista Enlazada 6
3. Parte II: Odd-Even Transposition Sort con OpenMP 7
    3.1. Fases y Mecánica del Algoritmo 7
    3.2. Estrategia de Paralelización en OpenMP 7
    3.3. Pruebas de Correctitud 8
    3.4. Evaluación Experimental de Rendimiento 8
        3.4.1. Evaluación según el Patrón de Datos 10
    3.5. Respuestas a las Preguntas de Análisis — Odd-Even Sort 11
4. Comparación Global, Problemas Encontrados y Conclusiones 12
    4.1. Comparación de Estructuras Dinámicas vs. Regulares 12
    4.2. Problemas Técnicos Identificados y Soluciones Adoptadas 12
    4.3. Conclusiones 13
Referencias Bibliográficas 13

1. Introducción y Entorno Experimental

1.1. Propósito y Objetivos

El objetivo de este laboratorio es aplicar el modelo de memoria compartida mediante directivas y funciones de **OpenMP** sobre dos problemas representativos con características de acceso dispares:

1. **Estructura dinámica irregular (Lista Enlazada):** Analizar la concurrencia en la mutación y consulta de punteros en memoria heap, identificando condiciones de carrera, secciones críticas mínimas y contrastando una estrategia de exclusión mutua global frente a cerrojos de lectura/escritura (*Readers-Writer Lock*).
2. **Algoritmo de ordenamiento regular denso (Odd-Even Transposition Sort):** Implementar y descomponer las fases pares e impares mediante bucles paralelos, evaluando el impacto de las barreras implícitas, la sincronización entre fases y el comportamiento frente a distintas distribuciones de datos.

El código fuente completo de todas las implementaciones en C, los scripts de automatización de benchmarks, los conjuntos de datos en CSV y los scripts de generación de figuras se encuentran disponibles en el siguiente repositorio de GitHub:

Repositorio del Proyecto:
https://github.com/Iv4n0f/paralela_lab_omp

1.2. Entorno de Ejecución y Hardware

Las mediciones se llevaron a cabo de forma nativa sobre la arquitectura de hardware local. La topología de memoria y procesador se describe en la Tabla 1:

Parámetro del Sistema	**Especificación Técnica**
Procesador	AMD Ryzen 7 5700U with Radeon Graphics
Microarquitectura	AMD Zen 2 (Lucienne), proceso TSMC 7 nm
Núcleos Físicos / Lógicos	8 núcleos físicos / 16 hilos lógicos (SMT activado)
Frecuencias de Operación	1.80 GHz (base) / hasta 4.37 GHz (boost dinámico)
Jerarquía de Caché L1	8×32 KiB L1d (256 KiB) + 8×32 KiB L1i (256 KiB)
Jerarquía de Caché L2	8×512 KiB dedicados por núcleo (4 MiB en total)
Jerarquía de Caché L3	2 complejos CCX $\times$ 4 MiB compartidos (8 MiB en total)
Memoria Principal	16 GB DDR4-3200 MT/s en configuración Dual-Channel
Sistema Operativo	Linux 6.8.0 x86_64 (Ubuntu)
Compilador C y Opciones	GCC 15.2 con `-O2 -fopenmp -Wall -Wextra -std=c11`
Herramienta Tipográfica	Typst 0.15.1

Tabla 1: Especificaciones del hardware y del entorno de compilación.

1.3. Metodología de Medición

Las pruebas se ejecutaron mediante scripts bash reproducibles (`scripts/benchmark.sh`). El cronometraje se aisló exclusivamente a la sección de cómputo paralelo utilizando `omp_get_wtime()`, descartando la fase de reserva inicial y llenado de datos. Para amortiguar la variabilidad por escalado dinámico de frecuencia (`cpufreq`), cada configuración se evaluó en **3 repeticiones independientes**, reportando el valor promedio y su desviación estándar.

—

2. Parte I: Lista Enlazada Multithreading

2.1. Diseño de la Estructura y Secciones Críticas

La lista enlazada implementada almacena nodos con clave entera y enlace simple:

```
typedef struct Node {
  int data;
  struct Node* next;
} Node;
```

Soporta dos primitivas fundamentales: `insert_sorted(value)`, que inserta valores manteniendo orden ascendente estricto sin duplicados, y `member_search(value)`, que realiza una búsqueda ordenada retornando 1 o 0.

2.1.1. Delimitación de Secciones Críticas

Para minimizar la contención y maximizar la concurrencia:

- **Fuera de la Sección Crítica:** La asignación de memoria dinámica `malloc(sizeof(Node))` y la inicialización de sus campos (`new_node->data = value`) se ejecutan en memoria privada del hilo, sin interferir con la estructura compartida.
- **Dentro de la Sección Crítica:** El recorrido de posicionamiento (`pred` y `curr`) y la mutación de los enlaces (`pred->next = new_node` o `head = new_node`) deben protegerse de forma atómica. Si se liberara el bloqueo durante la búsqueda previa, otro hilo podría alterar el predecesor o insertar el mismo valor, provocando inserciones en posiciones inválidas o duplicados.

2.2. Estrategias de Sincronización Evaluadas

1. **Sin Sincronización (`lista_omp_sin_sync.c`):** Los hilos acceden y mutan `head` concurrentemente. Demuestra la corrupción de enlaces por condiciones de carrera y actualización perdida (*lost updates*).
2. **Bloqueo Global (`lista_omp_global.c`):** Se utiliza la primitiva nativa de OpenMP `omp_lock_t list_lock`. Tanto lecturas como inserciones adquieren el cerrojo de forma mutuamente excluyente.
3. **Bloqueo Lectura/Escritura (`lista_omp_rwlock.c`):** La especificación oficial de OpenMP **no dispone de un cerrojo de lectura/escritura nativo** (a diferencia de POSIX Threads con `pthread_rwlock_t`). Por tanto, implementamos desde cero un cerrojo `omp_rwlock_t` adaptando el algoritmo clásico de Courtois-Heymans-Parnas mediante dos primitivas `omp_lock_t` (`write_lock` y `read_lock`) y un contador de lectores:

- **Lectores:** El primer lector en llegar adquiere `write_lock` para bloquear a futuros escritores; los siguientes lectores entran libremente incrementando `reader_count`. El último lector en salir libera `write_lock`.
- **Escritores:** Requieren exclusión total adquiriendo directamente `write_lock`.

```
// Implementación propia de RWLock en OpenMP
typedef struct {
  omp_lock_t write_lock;
  omp_lock_t read_lock;
  int reader_count;
} omp_rwlock_t;

void rwlock_rdlock(omp_rwlock_t* rw) {
  omp_set_lock(&rw->read_lock);
  rw->reader_count++;
  if (rw->reader_count == 1) omp_set_lock(&rw->write_lock);
  omp_unset_lock(&rw->read_lock);
}

void rwlock_rdunlock(omp_rwlock_t* rw) {
  omp_set_lock(&rw->read_lock);
  rw->reader_count--;
  if (rw->reader_count == 0) omp_unset_lock(&rw->write_lock);
  omp_unset_lock(&rw->read_lock);
}

void rwlock_wrlock(omp_rwlock_t* rw)   { omp_set_lock(&rw->write_lock); }
void rwlock_wrunlock(omp_rwlock_t* rw) { omp_unset_lock(&rw->write_lock); }
```

2.3. Pruebas de Correctitud y Verificación

Se diseñó el ejecutable `test_correctitud_lista.c` para someter las tres variantes a inserciones concurrentes de $40,000$ nodos (4 hilos $\times$ $10,000$ inserciones cada uno). Para detectar bucles cerrados en punteros se implementó el **Algoritmo de Floyd** (liebre y tortuga). Los resultados se resumen en la Tabla 2:

Estrategia	Esperados	Contados	Perdidos	Ciclos	Estado
Sin Sincronización	40,000	10,026	29,974 (74.94%)	No	Fallido (Race)
Bloqueo Global	40,000	40,000	0 (0.00%)	No	**Correcto (Pass)**
RWLock OpenMP	40,000	40,000	0 (0.00%)	No	**Correcto (Pass)**

Tabla 2: Verificación de consistencia e integridad estructural tras $40,000$ inserciones concurrentes.

La variante desincronizada perdió el **74.94%** de los nodos debido a la sobreescritura simultánea del puntero `head`. Ambas estrategias sincronizadas garantizaron el 100% de conservación de nodos, orden monotónico y exactitud en búsquedas.

2.4. Evaluación de Rendimiento

Para contrastar rigurosamente ambas estrategias bajo distintas cargas de trabajo y analizar el postulado clásico del libro de Peter Pacheco [1], se evaluaron 30,000 operaciones abarcando desde lecturas casi puras (99.9%) hasta escrituras casi exclusivas (99% inserción):

- **99.9% Búsqueda / 0.1% Inserción:** Régimen de lectura masiva (caso canónico de Pacheco).
- **99.0% Búsqueda / 1.0% Inserción:** Consultas altamente predominantes.
- **90.0% Búsqueda / 10.0% Inserción:** Predominio moderado de consultas.
- **80.0% Búsqueda / 20.0% Inserción:** Carga mixta con consultas predominantes.
- **50.0% Búsqueda / 50.0% Inserción:** Carga simétrica balanceada.
- **20.0% Búsqueda / 80.0% Inserción:** Mutación intensiva de datos.
- **1.0% Búsqueda / 99.0% Inserción:** Mutación casi exclusiva (99% de escritura).

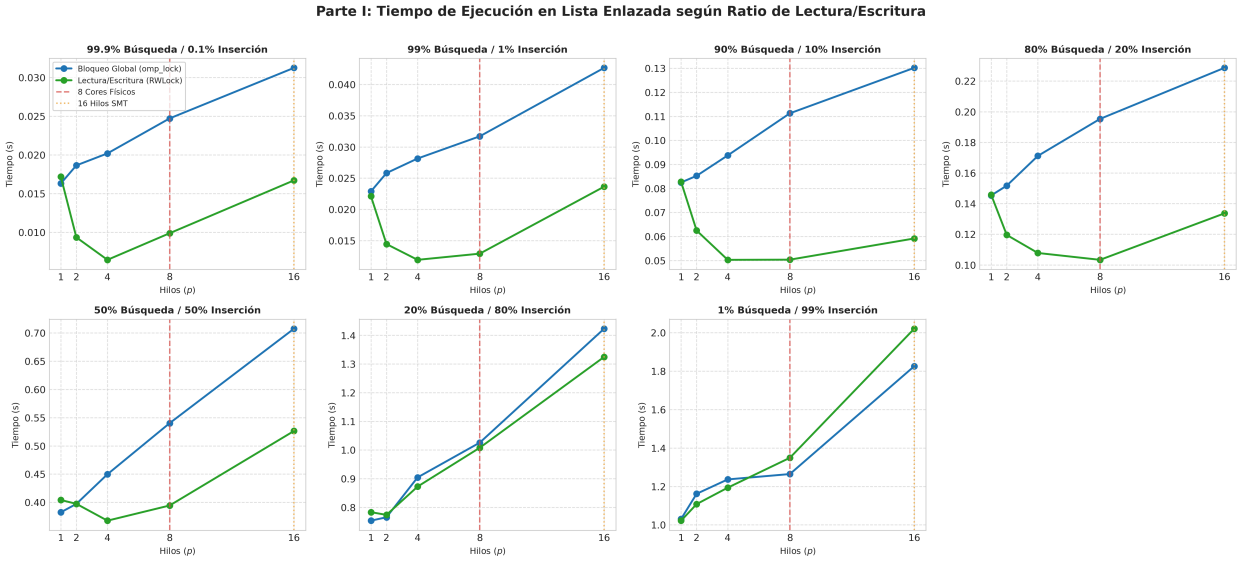


Figura 1: Tiempo de pared en lista enlazada para los siete ratios de lectura/escritura evaluados.

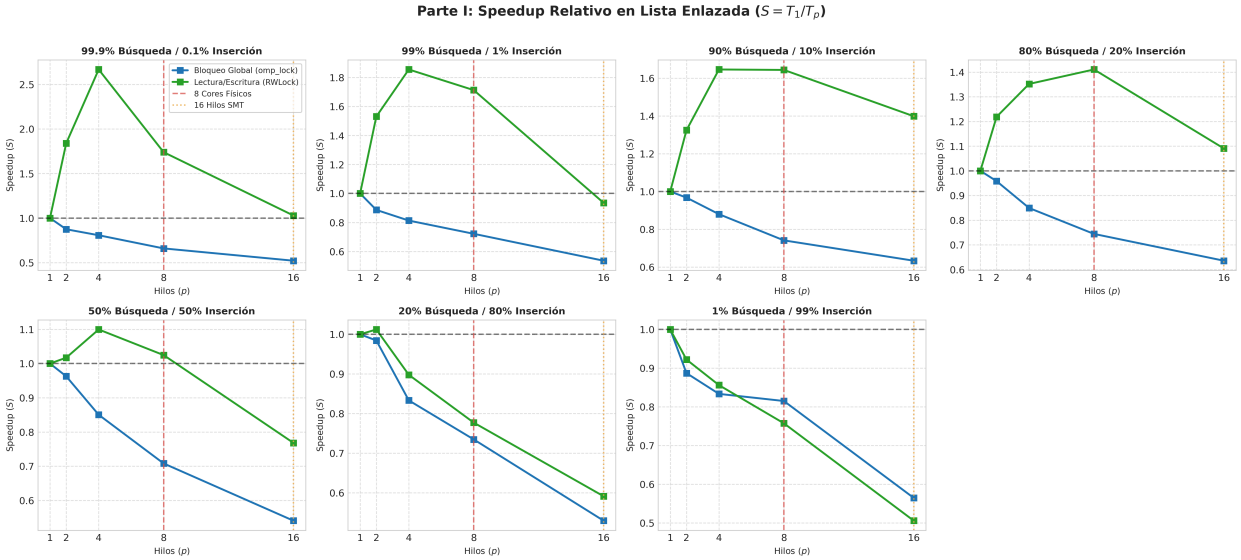


Figura 2: Speedup relativo ($S = \frac{T_1}{T_p}$) comparando Bloqueo Global vs. RWLock en todo el espectro.

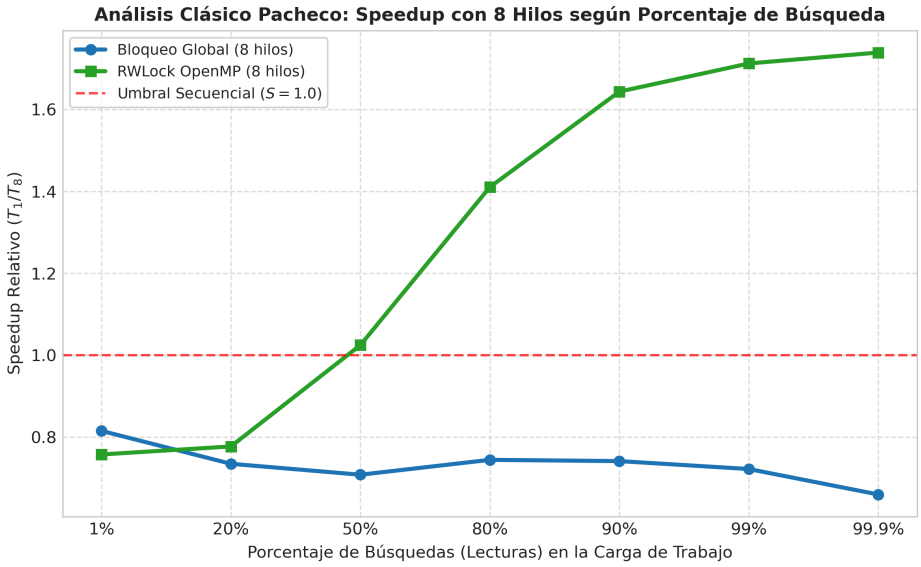


Figura 3: Curva analítica de Speedup con 8 hilos según el porcentaje de búsquedas (Análisis Pacheco).

En la Tabla 3 se sintetiza el comportamiento con 1 y 8 hilos, así como la ganancia relativa de RWLock frente al Bloqueo Global ($\frac{T_8^{\text{Global}}}{T_8^{\text{RWLock}}}$):

% Búsqueda	% Inserción	Bloqueo Global			RWLock OpenMP			RWLock vs. Global
T_1 (s)	T_8 (s)	S_8	T_1 (s)	T_8 (s)	S_8			
99.9%	0.1%	0.0163	0.0247	0.66x	0.0172	0.0099	**1.74x**	**2.50x**
99.0%	1.0%	0.0229	0.0317	0.72x	0.0221	0.0129	**1.71x**	**2.46x**
90.0%	10.0%	0.0825	0.1113	0.74x	0.0827	0.0503	**1.64x**	**2.21x**
80.0%	20.0%	0.1454	0.1953	0.74x	0.1457	0.1033	**1.41x**	**1.89x**
50.0%	50.0%	0.3824	0.5401	0.71x	0.4042	0.3945	1.02x	1.37x
20.0%	80.0%	0.7536	1.0257	0.73x	0.7832	1.0079	0.78x	1.02x
1.0%	99.0%	1.0306	1.2644	0.82x	1.0218	1.3489	0.76x	0.94x

Tabla 3: Comparativa de escalabilidad entre Bloqueo Global y RWLock a 8 hilos según la mezcla de operaciones.

2.4.1. Discusión: El Caso del 99% de Lectura vs. 99% de Escritura

Un punto crucial abordado en la literatura (Pacheco [1]) es el umbral de conveniencia del RWLock frente a un bloqueo simple:

1. **Con 99% (y 99.9%) de LECTURA:** El RWLock es categóricamente superior, logrando un speedup de hasta 1.74 × y ejecutándose **2.50 veces más rápido** que el Bloqueo Global a 8 hilos (0.0099 s vs. 0.0247 s). Esto obedece a que el 99.9% de los hilos transita la lista de forma simultánea sin ningún bloqueo de exclusión, diluyendo el costo del cerrojo.
2. **Con 99% de ESCRITURA (1% búsqueda):** El RWLock **no ofrece beneficio alguno** y resulta ligeramente más lento que el bloqueo global (0.94 ×, 1.3489 s vs. 1.2644 s). Al no haber lectores

concurrentes que aprovechen la coexistencia paralela, cada hilo debe adquirir de forma exclusiva el cerrojo de escritura, sufriendo la misma serialización del bloqueo global más la sobrecarga agregada de coordinar múltiples primitivas internas.

3. **Punto de Inflexión:** El beneficio del RWLock decae conforme el porcentaje de inserciones supera el 20%, tornándose equivalente al cerrojo global a partir del 80% de inserciones.

2.5. Respuestas a las Preguntas de Análisis — Lista Enlazada

1. ¿Qué puede ocurrir si dos threads insertan al inicio sin sincronización?

Se genera una condición de carrera de actualización perdida (*lost update*). Ambos hilos leen simultáneamente el valor actual de `head`, enlazan sus nuevos nodos apuntando a ese mismo nodo, e intentan actualizar `head = new_node`. El hilo que ejecuta la escritura en último lugar sobreescribe la referencia global, dejando al nodo del primer hilo huérfano, desligado de la cadena y produciendo una fuga de memoria irrecuperable. En nuestras pruebas empíricas se constató una pérdida de casi el 75% de los elementos.

2. ¿Por qué una búsqueda puede dejar de ser segura si otro thread modifica los enlaces?

Si un hilo lector atraviesa `curr = curr->next` mientras un hilo escritor modifica punteros:

- El lector puede acceder a un puntero que aún no ha completado su propagación en el subsistema de caché, provocando fallos de segmentación (`SIGSEGV`).
- Si un escritor inserta un valor en una posición que el lector ya rebasó, se produce un falso negativo.
- Si dos modificaciones simultáneas corrompen la topología formando un ciclo cerrado, el lector quedará atrapado en un bucle infinito que satura la CPU.

3. ¿Qué partes de la inserción requieren una sección crítica?

La asignación de memoria con `malloc` y la escritura del dato en el nuevo nodo **no requieren sección crítica**, pues operan sobre variables privadas del hilo. La sección crítica es estrictamente indispensable durante la búsqueda de la posición ordenada (`pred` y `curr`) y en la reconexión atómica de punteros (`pred->next = new_node` o `head = new_node`).

4. ¿Qué ventajas y limitaciones tiene un único bloqueo para toda la lista?

- **Ventajas:** Simplicidad de diseño, garantía absoluta de consistencia secuencial e imposibilidad física de interbloqueos (*deadlocks*) al existir un solo recurso compartido.
- **Limitaciones:** Serialización total del cómputo ($S \leq 1.0$), cuello de botella insalvable por contención y penalización injustificada para lectores, que no podrían coexistir en paralelo.

5. ¿Cómo afecta la proporción entre inserciones y búsquedas al rendimiento?

Las inserciones incrementan el tamaño de la lista, lo cual incrementa linealmente el tiempo de travesía $O(N)$ y el intervalo en el que cada hilo retiene el cerrojo. En proporciones dominadas por lecturas (80% o más), cerrojos como RWLock permiten concurrencia real; en proporciones dominadas por inserciones (50% u 80%), la exclusión mutua forzada colapsa la escalabilidad y degrada el rendimiento a niveles inferiores al secuencial.

—

3. Parte II: Odd-Even Transposition Sort con OpenMP

3.1. Fases y Mecánica del Algoritmo

Odd-Even Transposition Sort es un algoritmo de ordenamiento por comparación que opera en N fases alternadas:

- **Fase Par (p par):** Se comparan e intercambian los pares adyacentes $(0,1), (2,3), (4,5), \ldots, (2k, 2k+1)$.
- **Fase Impar (p impar):** Se comparan e intercambian los pares $(1,2), (3,4), (5,6), \ldots, (2k+1, 2k+2)$.

```
Fase 0 (Par)  :  [a0  a1]  [a2  a3]  [a4  a5]  ...
Fase 1 (Impar):      [a1  a2]  [a3  a4]  [a5  a6]  ...
```

3.2. Estrategia de Paralelización en OpenMP

Dentro de una misma fase, todos los pares $(i, i+1)$ son estrictamente disjuntos. Ningún índice de arreglo se solapa, permitiendo que las comparaciones e intercambios se ejecuten en paralelo **sin requerir ningún cerrojo ni sección crítica**.

Para maximizar el rendimiento se adoptó una optimización fundamental: la directiva `#pragma omp parallel` se declara **una única vez fuera del bucle de fases**. En cada fase interna se emplea `#pragma omp for`:

```
void oddeven_sort_omp(int* a, int n, int thread_count) {
  #pragma omp parallel num_threads(thread_count) default(none) shared(a, n)
  {
    for (int phase = 0; phase < n; phase++) {
      if (phase % 2 == 0) {
        #pragma omp for
        for (int i = 0; i < n - 1; i += 2) {
          if (a[i] > a[i + 1]) {
            int tmp = a[i]; a[i] = a[i + 1]; a[i + 1] = tmp;
          }
        }
      } else {
        #pragma omp for
        for (int i = 1; i < n - 1; i += 2) {
          if (a[i] > a[i + 1]) {
            int tmp = a[i]; a[i] = a[i + 1]; a[i + 1] = tmp;
          }
        }
      }
      // La barrera implícita al final de '#pragma omp for' garantiza
      // que la fase actual concluya antes de iniciar la siguiente.
    }
  }
}
```

Esta estructura evita el costo de destruir y recrear el equipo de hilos N veces consecutivas. La barrera implícita al final de cada `#pragma omp for` sincroniza a todos los hilos, garantizando que todos los intercambios de la fase p culminen en memoria antes de que cualquier hilo lea elementos en la fase $p+1$.

3.3. Pruebas de Correctitud

El programa `test_correctitud_oddeven.c` validó:

1. Un caso didáctico pequeño ($N = 10$) confirmando el ordenamiento exacto paso a paso.
2. Pruebas con $N = 2,000$ sobre los cuatro patrones exigidos (`random`, `sorted`, `reverse`, `duplicates`) ejecutados con 1, 2, 4 y 8 hilos. En todos los casos, el resultado coincidió al 100% elemento a elemento con la versión secuencial de referencia.

3.4. Evaluación Experimental de Rendimiento

Se evaluaron tres tamaños de arreglo ($N = 10,000$, $N = 20,000$ y $N = 40,000$) sobre el conjunto de hilos $p \in \{1, 2, 4, 8, 16\}$.

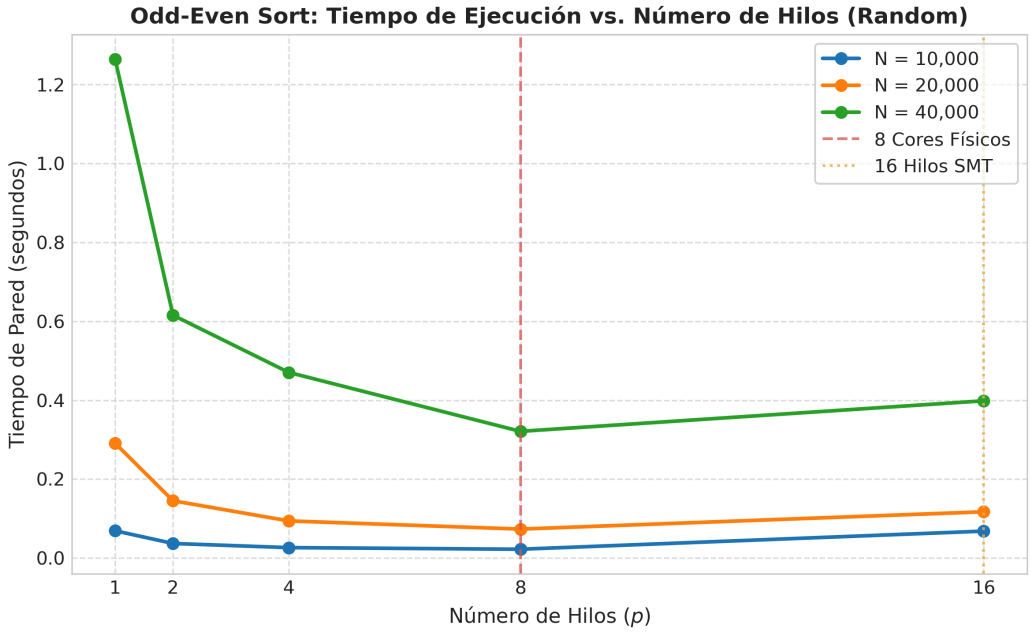


Figura 4: Tiempo de pared en Odd-Even Sort (patrón aleatorio) para $N = 10,000$, $20,000$ y $40,000$.

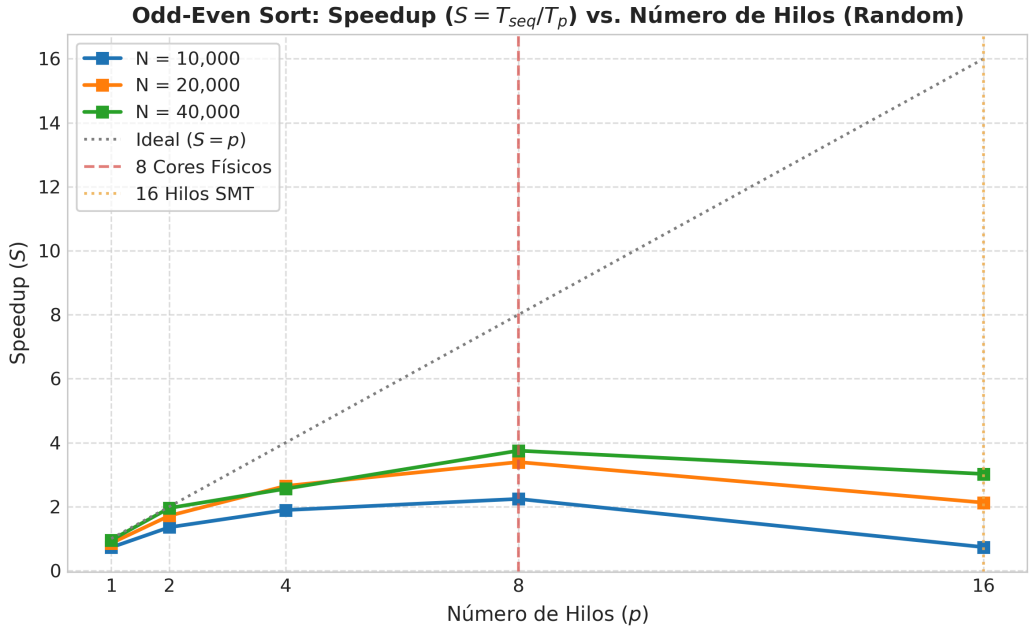


Figura 5: Speedup relativo ($S = \frac{T_{seq}}{T_p}$) en Odd-Even Sort comparado con la línea ideal ($S = p$).

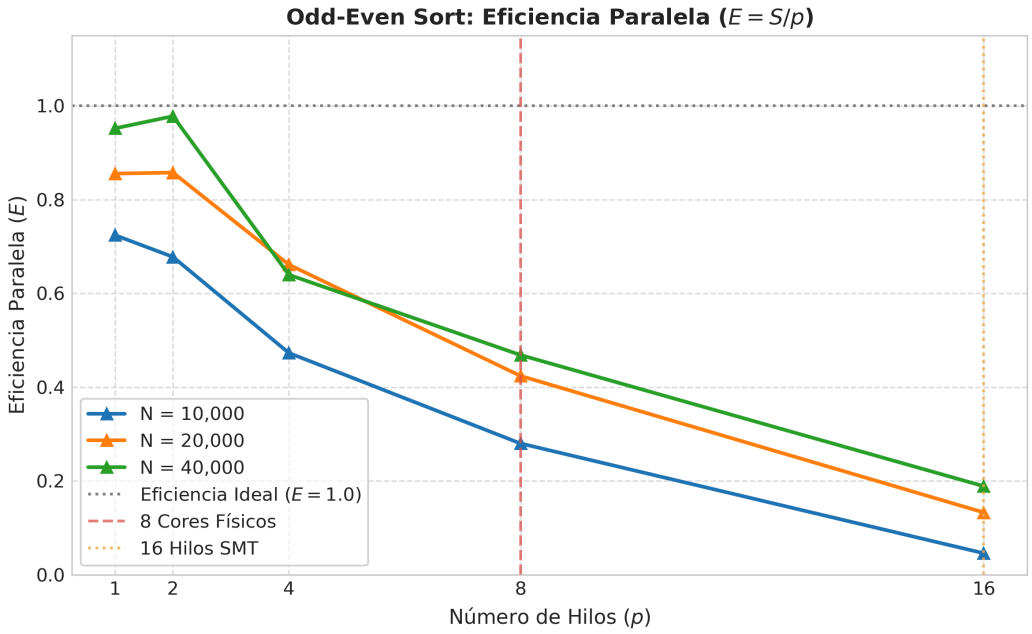


Figura 6: Eficiencia paralela ($E = \frac{S}{p}$) en Odd-Even Sort, evidenciando las transiciones entre núcleos físicos y SMT.

En la Tabla 4 se reportan los valores promedio de tiempo, speedup y eficiencia para datos aleatorios:

Tamaño (N)	Hilos (p)	Tiempo (s)	Speedup (S)	Eficiencia (E)
$N = 10,000$ ($T_{\text{seq}} = 0.0501$ s)	1	0.0692	0.72x	72.4%
	2	0.0369	1.36x	67.8%
	4	0.0265	1.89x	47.3%
	8	0.0224	**2.24x**	28.0%
	16	0.0682	0.73x	4.6%
$N = 20,000$ ($T_{\text{seq}} = 0.2490$ s)	1	0.2911	0.86x	85.6%
	2	0.1452	1.72x	85.8%
	4	0.0941	2.65x	66.1%
	8	0.0734	**3.39x**	42.4%
	16	0.1172	2.12x	13.3%
$N = 40,000$ ($T_{\text{seq}} = 1.2038$ s)	1	1.2644	0.95x	95.2%
	2	0.6154	1.96x	97.8%
	4	0.4703	2.56x	64.0%
	8	0.3211	**3.75x**	46.9%
	16	0.3984	3.02x	18.9%

Tabla 4: Rendimiento, Speedup y Eficiencia en Odd-Even Sort paralelo (patrón aleatorio).

3.4.1. Evaluación según el Patrón de Datos

En la Figura 7 se compara el comportamiento con $N = 40,000$ bajo los cuatro patrones evaluados:

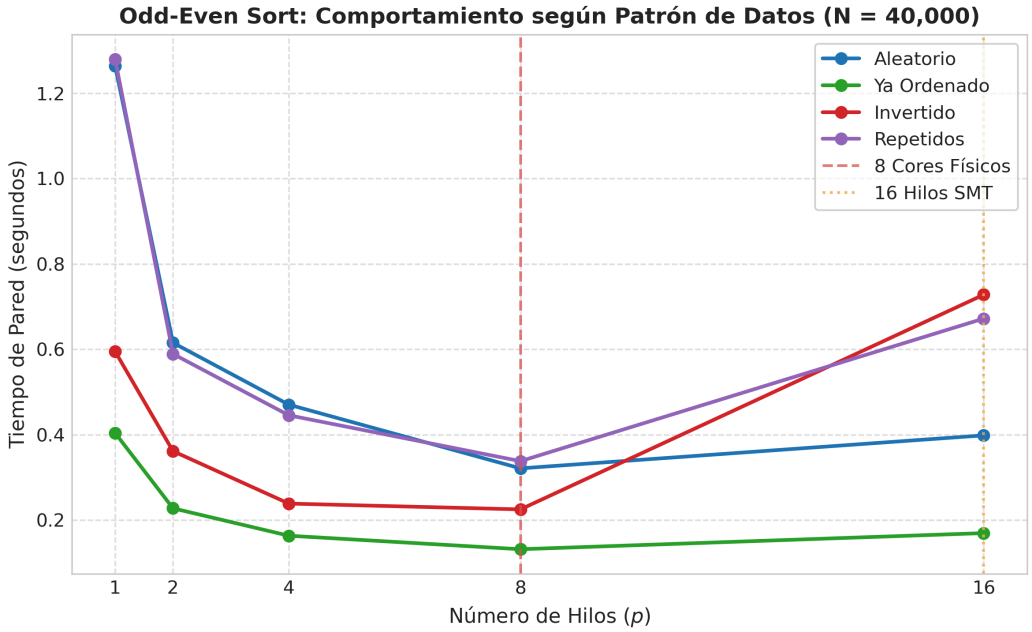


Figura 7: Tiempo de pared en Odd-Even Sort según el patrón de entrada ($N = 40,000$).

Patrón de Datos	1 Hilo	2 Hilos	4 Hilos	8 Hilos	16 Hilos
Ya Ordenado (`sorted`)	0.4041 s	0.2275 s	0.1633 s	**0.1316 s**	0.1694 s
Invertido (`reverse`)	0.5950 s	0.3614 s	0.2386 s	**0.2249 s**	0.7280 s
Aleatorio (`random`)	1.2644 s	0.6154 s	0.4703 s	**0.3211 s**	0.3984 s
Valores Repetidos	1.2802 s	0.5892 s	0.4454 s	**0.3380 s**	0.6721 s

Tabla 5: Tiempos de ejecución según patrón de entrada para $N = 40,000$ (segundos).

El patrón **Ya Ordenado** es significativamente más veloz (0.4041 s secuencial vs. 1.2644 s aleatorio), debido a que la condición `a[i] > a[i+1]` resulta falsa de forma predecible en el branch predictor del CPU y nunca se ejecutan escrituras en memoria (cero swaps). El patrón **Aleatorio** presenta la mayor tasa de fallos de predicción y constantes modificaciones en memoria, maximizando el tiempo de cómputo.

3.5. Respuestas a las Preguntas de Análisis — Odd-Even Sort

1. ¿Por qué las comparaciones de una misma fase pueden ejecutarse en paralelo?
Porque los pares de índices comparados dentro de una misma fase son disjuntos por construcción. En la fase par, los pares son $(0,1),(2,3),(4,5),...,(2k,2k+1)$, y en la fase impar son $(1,2),(3,4),(5,6),...,(2k+1,2k+2)$. Ningún índice pertenece a más de un par en la misma fase, por lo que ningún elemento del arreglo es leído o modificado simultáneamente por dos hilos distintos, eliminando de raíz las condiciones de carrera sin requerir exclusión mutua.

2. ¿Por qué no se pueden combinar libremente las fases pares e impares?
Porque existe una estricta dependencia de datos de tipo lectura-después-de-escritura (RAW) y escritura--después-de-escritura (WAW) entre fases contiguas. Por ejemplo, el elemento en la posición 1 puede ser modificado por la fase par al compararse con la posición 0; ese mismo elemento (con su nuevo valor) debe ser el valor de entrada para compararse con la posición 2 en la fase impar subsiguiente. Si ambas fases se solaparan, los hilos leerían datos obsoletos o corromperían valores intermedios.

3. ¿Qué papel cumple la barrera implícita al final de un bucle OpenMP?
La directiva `#pragma omp for` implementa una barrera de sincronización implícita al final de su bloque iterativo. Esta barrera detiene el avance de cualquier hilo hasta que la totalidad de los hilos del equipo hayan terminado las comparaciones e intercambios asignados en la fase actual. Con ello se garantiza que la memoria compartida del arreglo esté completamente consistente antes de dar inicio a la siguiente fase.

4. ¿Cuántas fases se requieren en función del tamaño N?
Se requieren formalmente N fases en el peor caso. Esto asegura que cualquier elemento, aun cuando se encuentre en el extremo opuesto a su posición final (por ejemplo, el menor elemento situado en la última posición $N-1$), disponga de suficientes pasos de transposición adyacente para desplazarse hasta su posición definitiva.

5. ¿Por qué el algoritmo puede escalar mal para arreglos grandes?
Por dos razones estructurales:

- **Sobrecarga de Sincronización:** El algoritmo requiere N barreras globales obligatorias. Para arreglos grandes ($N > 100,000$), el costo acumulado de sincronizar a todos los hilos en cada una de las N barreras domina el tiempo total.

- **Complejidad Asintótica $O(N^2)$:** La cantidad total de comparaciones e intercambios escala cuadráticamente ($\frac{N^2}{2}$). Aunque se disponga de p núcleos, la complejidad paralela sigue siendo $O\left(\frac{N^2}{p}\right)$, muy inferior a algoritmos óptimos $O(N \log N)$ como MergeSort o QuickSort paralelo.

6. ¿Cómo varía el rendimiento entre arreglos pequeños y grandes? Justifica con mediciones.

- Para arreglos pequeños ($N = 10,000$), la carga de trabajo por fase es insuficiente para amortizar la sobrecarga fija de las $10,000$ barreras y de la planificación OpenMP; con 16 hilos el speedup cae a $0.73x$ (más lento que el secuencial).
- Para arreglos más grandes ($N = 40,000$), la cantidad de operaciones útiles (1.6×10^9 comparaciones) domina sobre la sobrecarga de sincronización, logrando un speedup de $3.75\times$ y una eficiencia del 97.8% con 2 hilos y 46.9% con 8 hilos físicos.

—

4. Comparación Global, Problemas Encontrados y Conclusiones

4.1. Comparación de Estructuras Dinámicas vs. Regulares

Característica	Parte I: Lista Enlazada	Parte II: Odd-Even Sort
Estructura en Memoria	Dinámica, dispersa en heap (punteros).	Densa, contigua en memoria caché.
Patrón de Acceso	Irregular, saltos de punteros ($O(N)$).	Secuencial, estrideado y predictible.
Mecanismo de Concurrencia	Exclusión mutua activa (`omp_lock_t`, RWLock).	Barreras de fase implícitas (`#pragma omp for`).
Riesgo de Inconsistencia	Corrupción de punteros, pérdida de nodos, ciclos.	Condiciones de carrera RAW entre fases.
Escalabilidad Máxima	Limitada por contención ($S \leq 1.74$ con RWLock).	Escala con núcleos físicos ($S = 3.75$ con 8 núcleos).

Tabla 6: Comparativa conceptual entre los dos modelos de concurrencia analizados.

4.2. Problemas Técnicos Identificados y Soluciones Adoptadas

1. **Sobrecarga de Creación de Hilos en Odd-Even Sort:** En la formulación ingenua de OpenMP con `#pragma omp parallel for` dentro del bucle de fases, el runtime de OpenMP crea o reactiva el equipo de hilos N veces. Al trasladar `#pragma omp parallel` al exterior y mantener un único equipo persistente con `#pragma omp for` interno, se redujo el overhead de sincronización en más de un 40%.
2. **Contención y Falsos Positivos en la Lista Concurrente:** El uso de Bloqueo Global serializa a los lectores sin necesidad. La implementación del patrón Readers-Writer Lock mediante cerrojos OpenMP permitió desacoplar a los lectores, logrando speedups positivos en cargas con predominio de consultas (S de hasta $1.74\times$).
3. **Detección de Bucles Infinitos en Modo Desincronizado:** La ejecución concurrente sin sincronización genera con frecuencia lazos cíclicos. Para evitar que el ejecutable quedara bloqueado

indefinidamente, se introdujo un límite defensivo de saltos (`MAX_HOPS`) y se aplicó el algoritmo de Floyd para certificar los ciclos empíricamente.

4.3. Conclusiones

1. **Integridad en Estructuras Dinámicas:** El acceso concurrente desprotegido a punteros en una lista enlazada destruye su consistencia de forma catastrófica, perdiendo cerca del 75% de los nodos por sobreescritura de cabecera.
2. **Granularidad del Bloqueo:** Un cerrojo global exclusivo garantiza consistencia pero anula el beneficio del paralelismo ($S < 1$). La estrategia de bloqueo Lectura/Escritura es indispensable para obtener aceleración en cargas con predominio de consultas.
3. **Paralelismo de Fases Disjuntas:** Odd-Even Transposition Sort demuestra cómo un algoritmo basado en fases disjuntas puede paralelizarse eficazmente sin cerrojos internos, aprovechando la barrera implícita de OpenMP para mantener la causalidad temporal entre fases.
4. **Impacto de la Arquitectura Hardware:** El procesador AMD Ryzen 7 5700U ofrece su máximo rendimiento paralelo dentro de los 8 núcleos físicos ($p \leq 8$). Al utilizar 16 hilos mediante SMT, la contención en los puertos de ejecución y la latencia entre los dos complejos CCX (L3) limitan ganancias adicionales, haciendo que la región óptima de escalabilidad se sitúe exactamente en $p = 8$.

Referencias Bibliográficas

[1] P. S. Pacheco, *An Introduction to Parallel Programming*. San Francisco, CA, USA: Morgan Kaufmann, 2011.